

Spojená škola, Komárňanská 28, 940 75 Nové Zámky
o. z. Stredná priemyselná škola elektrotechnická S. A. Jedlika –
Jedlik Ányos Elektrotechnika Szakközépiskola

„Kartové hrady – vývoj hier s tematikou tower defense“

„Kártyavárak – tower defense játék fejlesztése“

PRAKTICKÁ ČASŤ ODBORNEJ ZLOŽKY MATURITNEJ SKÚŠKY
VLASTNÝ PROJEKT

A SZAKVIZSGA GYAKORLATI RÉSZÉ
ÖNÁLLÓ TERVEZET

Nové Zámky

2025/2026

Ročník: **štvrtý**

Évfolyam: **negyedik**

Konzultant / Konzulens:

Žiak/Diák:

Gabriel Lizsica

Mgr. Miklós Vontszemű

Spojená škola, Komárňanská 28, Nové Zámky, 940 75
Stredná priemyselná škola elektrotechnická S.A. Jedlika
Jedlik Ányos Elektrotechnikai Szakközépiskola

PRAKTICKÁ ČASŤ ODBORNEJ ZLOŽKY MATURITNEJ SKÚŠKY
VLASTNÝ PROJEKT
A SZAKVIZSGA GYAKORLATI RÉSZÉ
ÖNÁLLÓ TERVEZET

Meno žiaka:	Gabriel Lízica
Tanuló neve:	
Trieda:	IV.DI
Osztály:	
Školský rok:	2025/2026
Iskolaév:	
Študijný odbor:	Informačné a sieťové technológie
Tanulmányi szak:	
Zameranie:	Programovanie
Szakirányzat:	
Interný konzultant:	Mgr. Miklós Vontszemű
Belső konzulens:	
Externý konzultant:	
Külső konzulens:	
Názov projektu	Kártyavárak – tower defense játék fejlesztése
A feladat megnevezése:	

žiak / diák

externý konzultant / külső konzulens

interný konzultant / belső konzulens

zástupkyňa riaditeľky školy / igazgató helyettes

V Nových Zámkoch 14.10.2025/2025.10.14

Spojená škola, Komárňanská 28, Nové Zámky, 940 75
Stredná priemyselná škola elektrotechnická S.A. Jedlika
Jedlik Ányos Elektrotechnikai Szakközépiskola

Čiastkové úlohy

Részfeladatok:

- 1. Výber a nastavenie vhodného vývojového prostredia - Megfelelő fejlesztési felület kiválasztása és beállítása.**
- 2. Definovanie a rozvíjanie herného konceptu - Játékmenet meghatározása és kidolgozása.**
- 3. Vytvorenie používateľského rozhrania - Kezelőfelület elkészítése.**
- 4. Automatické generovanie herného pola pomocou 2D grid systemu - A játékterület automatikus generálása 2D grid rendszer segítségével.**
- 5. Správa herných stavov a herných kariet - Játékállapotok és játékkártyák kezelése.**
- 6. Vytvorenie systému ukladania - Mentési rendszer kialakítása.**
- 7. Testovanie a oprava chýb - Tesztelés és hibák javítása.**

Becsületbeli nyilatkozat

Alulírott.....születettkinyilatkozo
m és aláírással igazolom, hogy a gyakorlati érettségi munkámat egyedül belső/külső
konzulensem irányítása alatt dolgoztam ki.

diák aláírása

Köszönetnyilvánítás

Szeretném megköszönni belső konzulensemnek,
a feladatom elkészítéséhez és kidolgozásához nyújtott szaktanácsokat és segítséget,

diák aláírása

TARTALOM

BEVEZETÉS	8
1 A FELADAT ISMERTETÉSE	10
1.1 A játékmotorok	10
1.2 A Unity játékmotor	11
1.3 A C# programozási nyelv	13
2 A FELADAT CÉLJA.....	14
3 A FELADAT KIDOLGOZÁSÁNAK MENETE	15
3.1 Fejlesztési környezet kiválasztása.....	15
3.2 Elméleti háttér és műfaj	16
3.3 A játéktér véletlenszerű létrehozása.....	17
3.4 Adatkezelés és optimalizálás	22
3.5 Felhasználói felület (GUI)	27
4 A FELADAT KIDOLGOZÁSÁNAK EREDMÉNYEI.....	32
5 ÉRTEKEZÉS	33
6 A FELADAT ÉRTÉKELÉSE.....	34
6.1 A célkitűzések teljesülésének vizsgálata	34
6.2 Gyakorlati hasznosítás és jövőbeni perspektívák	34
ÖSSZEFOGLALÁS.....	35
RESUME.....	36
FELHASZNÁLT IRODALOM	38

Képjegyzék

ábra 1: A jelenet végső hierarchiája	16
ábra 2: CardTextureScriptableObject szkript implementációja	23
ábra 3: EnemyScriptableObject szkript implementációja.....	23
ábra 4: PlayerProfileScriptableObject szkript implementációja.....	24
ábra 5: SpellScriptableObject szkript implementációja (példa)	24

BEVEZETÉS

Gyerekkorom óta fontos szerepet játszanak az életemben a videojátékok. Nemcsak szórakozásként, hanem olyan tevékenységként, ami kikapcsol, ugyanakkor gondolkodásra is készítet. Legközelebb hozzám a stratégiai játékok állnak, mivel ezek nem csak ügyességet, hanem tervezést, logikus gondolkodást és előrelátást is igényelnek. Az évek során sok különféle műfajjal találkoztam, és mindig érdekesnek találtam, amikor egy játék valamilyen új, addig nem igazán látott, ötleten alapult. Így jött az ötlet, hogy én magam is megpróbáljak létrehozni egy saját játékot – olyat, ami ötvözi azokat az elemeket, amiket a legjobban szeretek.

A projekt alapötlete két különböző játék típus, a tower defense és a kártyajáték, ötvöztetésével született. Annak ellenére, hogy mindkét műfaj önállóan jól ismert, a kettő ötvözte még viszonylag ritka. Ez számomra nemcsak kihívást jelentett, hanem lehetőséget is arra, hogy valami újat alkossak. A tower defense játékokban a taktikus elhelyezkedés és a tervezés számít, míg a kártyajátékokban a pakli összeállítása és a megfelelő pillanatban való lap kijátszás. A kihívás az, hogy ezt a két rendszert úgy kell összeállítani, hogy azok ne ütközzenek egymással, egyik se szoruljon háttérbe és a játék élvezhető legyen.

A munka célja egy olyan játék elkészítése, amely egyszerre gondolkodtat és szórakoztat. Fontosnak tartottam, hogy a játék valami újat nyújtson a játékosnak, miközben megtartja a klasszikus stratégiai játékok elemeit. Olyan rendszert akartam kidolgozni, saját maga alakíthatja ki a stratégiáját. A cél nem csupán az volt, hogy technikailag működő játékot készítek, hanem hogy az hosszú távon is érdekes maradjon. Illetve szerettem volna, ha a játék nem válik unalmassá vagy kiszámíthatóvá hosszabb távon sem. Ezt úgy próbáltam elérni, hogy véletlenszerű elemeket is beépítettem, például a pályák generálásába vagy a szörnyek hullámaiba. Így minden egyes játék más élményt nyújthat, és más stratégiát igényel, még akkor is, ha a játékos ugyanazokat a kártyákat használja.

A projekt során nem csak játékfejlesztéssel foglalkozom, hanem azzal is, hogy hogyan lehet egy játékmenetet izgalmassá és változatossá tenni hosszú távon. A probléma az újrajátszhatóság és a stratégiai mélység fenntartása egy végtelen játékmenetű játékban. Hogyan lehet úgy kialakítani egy játékrendszert, hogy az mindig kínáljon új kihívásokat, ugyanakkor ne váljon túl bonyolulttá vagy túlzottan kiszámíthatatlanná. Illetve az is fontos volt, hogy a játékosnak szabadsága legyen saját döntéseket hozni. A különböző kártyák kombinációi, fejlesztései, és a pakli összetételének meghatározása, mind lehetőséget adnak a saját játéktílus és stratégia kialakítására. A véletlenszerűen generált szintek és a változó

ellenségtípusok pedig biztosítják, hogy ne ismétlődjenek meg ugyanazok a helyzetek. Így a gondolkodtató is marad, még sokadik végigjátszás után is.

Fontos volt számomra az is, hogy a játék döntéseket is követeljen meg. Ezért lett a középpontba helyezve a kártyarendszer. A játékos maga választhatja ki, hogy milyen kártyákat használ, milyen stratégiát követ. A pakli összeállítása, a kártyák fejlesztése és kijátszása mind teljes mértékben a játékos döntésein múlik. Ezzel szerettem volna elérni, hogy mindenki a saját stílusában játszhasson. Így nemcsak egy előre meghatározott útvonalat lehet követni, hanem saját taktikákat és megoldásokat lehet kidolgozni. A kártyák fejlesztése és vásárlása hosszabb távú célokat is ad, ami segít abban, hogy a játékos motivált maradjon, és érezze a fejlődést minden egyes játék során. Az, hogy a játék gyakorlatilag végtelen, újabb lehetőséget ad arra, hogy mindenki a saját tempójában fedezze fel és fejlessze tovább a rendszert.

1 A FELADAT ISMERTETÉSE

1.1 A játékmotorok

A játékmotor (game engine) olyan komplex szoftver rendszer, amely digitális játékok fejlesztéséhez biztosít integrált fejlesztői környezetet, futtatókörnyezetet és szolgáltatás réteget. A játékmotor egy úgynevezett absztrakciós réteg az operációs rendszer, a hardver (CPU, GPU, memória, bemeneti eszközök) és a játéklogika között. Ezzel lehetővé teszi a fejlesztők számára, hogy a játékmeneti és tartalmi kérdésekre koncentráljanak az alacsony szintű technikai részletek helyett. [1, 4]

A játékmotor tulajdonképpen egy szoftverrendszer, amely biztosítja a valós idejű grafikus megjelenítést (renderelés), kezeli a fizika valósághű szimulációját, alkalmazza a bemeneti- és kimeneti rendszereket, támogatja a hangfeldogozást, kezeli az erőforrásokat (asset management), és gyakran tartalmaz beépített animációs vagy hálózati rendszereket. A renderelő alrendszer felelős a grafikai objektumok valós idejű ábrázolásáért. Fő feladatai a geometria feldolgozása (vertex feldolgozás) és abból adódó alfeladatok, mint például az árnyékolás, a fény- és árnyék szimuláció vagy a textúrázás. Ide tartozik a "Ray Tracing" is, amely a fénycsónák egyenkénti szimulálásával éri el a kívánt hatást, az élethű képet. [1, 4]

A fizikai alrendszer szimulálja a valós világ fizikai törvényeit egy virtuális környezetbe való leegyszerűsített módon. A fizikai objektumok mozgását az úgynevezett merevtest-dinamika (rigid body dynamics) alapján szimulálja, gravitációval és a különböző erőhatásokkal együtt. Ez az alrendszer kezeli a testek közötti ütközések detektálása és azok fizikai következményeit is. A modul ezt matematikai egyenletek (differenciálegyenletek) lépésenkénti közelítő megoldásával teszi. Ha ugyanazokat a kiindulási adatokat adjuk meg, az eredmény mindig ugyanaz (determinista), vagy majdnem ugyanaz (kvázi-determinisztikus), mert a számítás apró kerekítései minimálisan eltérhetnek. [1, 4]

A játékmotor minden esetben biztosít lehetőséget magas szintű programozásra C++, C#, Lua vagy más magas szintű programozási nyelvek használatával. Maga a program architektúrája eseményvezérelt, komponens-alapú entitásrendszerrel (ECS - Entity-Component System). Az entitások magukban csak úgynevezett címkék. Ezeknek az entitásoknak vannak komponenseik, amelyek lehetnek adattároló struktúrák vagy ezeket az adatokat felhasználó, funkcionalitást implementáló szkriptek. [1, 4]

A legtöbb játékmotor tartalmaz beépített animációs vagy hálózati rendszert is. [1, 4]

1.2 A Unity játékmotor^[2, 3]

A Unity egy többplatformos, integrált fejlesztőkörnyezetet (IDE) és futtatókörnyezetet biztosító, komponens-alapú játékmotor. A szoftvert a Unity Technologies fejleszti és tartja karban.

A Unity architektúrájában az alapvető szervezési egység (entitás) a GameObject. Ehhez különböző komponensek sorolhatók, ezzel követve a korábban említett ECS architektúrát. Az elsődlegesen használt nyelv a C#, amely a .NET/Mono futtatókörnyezetre épül. Alapvetően a programozás objektumorientált modellben (OOP - Object Oriented Programming) valósul meg. A fejlesztő által létrehozott legtöbb osztály a rendszer által biztosított "MonoBehaviour" osztályból van származtatva. Ez az osztály biztosít beépített eseményvezérelt életciklus-metódusokat. A Unity újabb verzióiban megjelenik a DOTS (Data-Oriented Technology Stack), amely egy adatorientált programozási modellt tesz lehetővé.

A Unity alapvetően többféle renderelési architektúrát támogat és állít rendelkezésünkre, attól függően, hogy mire van szükségünk.

- Built-in Render Pipeline: klasszikus, alapszintű megközelítés.
- Universal Render Pipeline (URP): széles körben használt, legrugalmasabb. A legtöbb projekt esetében megfelelő.
- High Definition Render Pipeline (HDRP): kifejezetten nagy grafikai realizmusú és grafikai megterhelő projektekhez készült. Alkalmaz fizikai alapú renderelést (PBR - Physics Based Rendering), valós idejű globális megvilágítást és úgynevezett screen space effekteket.

Ezen kívül a motor támogatja a modern grafikus API-okat (OpenGL, DirectX, Vulkan, Metal) és kihasználja a GPU-alapú shader programozást, ShaderLab és HLSL alapú shader programozást is lehetővé tesz. A Shader Graph egy magas szintű absztrakciója a hagyományos shader kódolásnak. Ez egy vizuális, node alapú szerkesztést biztosít.

A Unity beépített fizikai motorja (3D esetében a PhysX, 2D esetében külön Box2D alapú fizikai rendszer) kezeli a merevtest dinamikát, ütközésetekálást, trigger rendszereket és az erő- és impulzus modelleket is. A fizikai frissítés adott időnként, azaz fix időlépésben történik (fixed timestep), amely biztosítja a determinisztikus viselkedést.

A Unity beépített animációs rendszere tartalmazza többek között a Mecanim animációs

rendszert, egy állapot alapú „Animator Controller” struktúrát és támogatást az úgynevezett inverz kinematikai animáláshoz. Ez mind lehetővé teszi komplex karakter animációk egyszerű és dinamikus vezérlését.

A Unity egyik legjelentősebb előnye a széleskörű platformtámogatás. Biztosít külön platformfüggő modulokat az applikáció adott platformra való építéséhez: Windows, macOS, Linux, Android, iOS, PlayStation, Xbox, Nintendo konzolok (Switch), tvOS, visionOS, WebGL és OpenXR. Az utóbbinak köszönhetően Unityn belül lehetőségünk van az ezt támogató eszközökre kiterjesztett valóság (Augmented Reality - AR) vagy virtuális valóság (Virtual Reality - VR) rendszereket építeni. E széleskörű támogatásnak köszönhetően a Unityt nem csak a játékiparban használják fel. Széles körben alkalmazott még ipari szimulációkban, építészeti vizualizációban, valamint filmipari virtuális produkcióban és digitális vizuális effektek létrehozásában. A valós idejű renderelési képesség és az interaktív modellezési lehetőség miatt a motor túlmutat a hagyományos videojáték fejlesztés keretein.

A Unity UI Toolkit a Unity motor modern felhasználói felület fejlesztésére szolgáló rendszere. A rendszer deklaratív megközelítése elválasztja a struktúrát, a megjelenést és a viselkedést. A struktúra UXML (Unity XML) nyelven van leírva, amely nagy mértékben hasonlít a HTML-hez. A megjelenítés stíluslapokkal, USS (Unity Style Sheets) segítségével történik, amely pedig nagy mértékben a CSS-hez hasonlít. A UI Toolkit vizuális fa (Visual Tree) alapú hierarchikus felépítést alkalmaz. A rendszer eseményvezérelt architektúrát használ, amely támogatja a buborékoló és az úgynevezett triggerelt eseménykezelést. A stílusok lehetővé teszik rezponzív és a felhasználó cselekedetire dinamikusan változó felületek kialakítását. A layout rendszer a Flexbox modellhez hasonló elrendezési mechanizmust használ. A UI Toolkit teljes mértékben integrált a Unity Editorba, így egyedi szerkesztőablakok és inspector nézetek is készíthetők vele. A rendszer támogatja az adatcsatlást, amely egyszerűbbé teszi az adatok és a felület szinkronizálását. A fejlesztő megadhatja, hogy az előre elkészített sablon melyik eleméhez milyen információ legyen kötve. Ezzel egyszerűsödik maga az adatok megjelenítése és azok változásakor a felhasználói felület frissítése. A UI Toolkit teljesítményoptimalizált, mivel retained-mode rendering megközelítést alkalmaz. A rendszerbe beépített UI Builder modul vizuális munkamenetet tesz lehetővé.

1.3 A C# programozási nyelv

A C# egy modern, többparadigmás, típusbiztos, objektumorientált programozási nyelv, amelyet a Microsoft fejlesztett ki a 2000-es évek elején a .NET platform részeként. Elsődleges célja egy olyan, erősen típusos, nagy teljesítményű és biztonságos nyelv megalkotása volt, amely ötvözi a C++ hatékonyságát a Java egyszerűbb memóriakezelési modelljével, miközben szoros integrációt biztosít a .NET futtatókörnyezettel. [4]

A C# statikusan típusos nyelv, ami azt jelenti, hogy a típusellenőrzés fordítási időben történik. A típusbiztonság csökkenti a futásidejű hibák valószínűségét. A típusinferencia (var) lehetővé teszi a fordító számára a típus automatikus meghatározását egy változó létrehozásakor. Ezt viszont csakis a következő esetekben lehet használni: ha a fordító a bal oldal (amikor van kezdőérték, tehát var használatánál kötelező a változó inicializálása), vagy referencia (például listából való lekérdezésnél vagy foreach ciklusban) alapján meg tudja határozni a típust. Ezen kívül var használata kimondottan kötelező anonim típusokál (var személy = new { Nev = „Anna”, Kor = 25}), mivel návtelenek, ezért nem is lehetne kírni a típusaikat. [5]

Mivel a C# egy objektumorientált nyelv, támogatja az OOP (objektum orientált programozás) négy allappillérét: absztrakció, enkapszuláció, öröklődés és polimorfizmus. Az osztály (class) az alapegység, ami tartalmazhat: mezőket (fields), tulajdonságokat (properties), metódusokat, eseményeket vagy indexelőket. [5]

A C# programok a Common Language Runtime (CLR) környezetben futnak, amely a .NET platform része. A fordító automatikus memóriakezelést alkalmaz. A szemétygyűjtő (Garbage Collector) felszabadítja azokat az objektumokat, amelyekre már nincs hivatkozás, ezzel csökkentve a memóriaszivárgás kockázatát. A C# elsődlegesen menedzselt (managed) környezetben működik, de lehetőség van nem menedzselt (unsafe) kód használatára is pointerekkel. [5, 6]

2 A FELADAT CÉLJA

A projekt központi témája egy olyan videójáték fejlesztése, amely két meghatározó stratégiai műfajt ötvözt: a tower defense (torony védelmi) és a kártyajáték (deck-builder) mechanizmusait.

A munka fő célja egy videójáték alapoktól való megtervezése és elkészítése. Fontos, hogy a játék tartalmazzon véletlenszerűen generált elemeket az újrajátszhatóság érdekében. Ilyen legyen a játéktér létrehozása és a kártyák húzása a pakliból. A játéktér minden egyes körben más legyen. Pontosabban az út, amely mentén az ellenségek eljutnak a játékos bázisához, az ellenségek kiindulópontja, és maga a bázis pozíciója lesz minden alkalommal más. A játékos a pakliját maga rakhassa össze a rendelkezésre álló kártyákból. A kör kezdetekor ezekből a játék automatikusan húz véletlenszerűen egy párat, majd a játék folyamán, ha a játékos elhasznál egyet, akkor minden alkalommal a játék automatikusan húz helyette egyet véletlenszerűen az előre megadott pakliból.

A játékmenetnek kiegyensúlyozottnak kell lenni. Fontos a játékos által használt kártyákat megfontoltan megtervezni. A könnyű fejlesztés, illetve a hatékonyabb memória takarékoság érdekében a kártyák adatai JSON fájllokba kell elmenteni. Ezeket a fájlokat a program betölti, majd a játékos elmentett adatai alapján kiolvassa betöltött információkból a szükséges adatokat. Ezeket az adatokat pedig tovább az adott kártyához tartozó scriptable object-nek. Amikor a játékos kijátszik egy kártyát, az adatok innen vannak kiolvasva.

Ki kell alakítani egy mentési rendszert, amelyen keresztül a játékos bármikor elmentheti az aktuális adatait. Ilyen, hogy melyik kártyából mennyi van és, hogy azok hányas szintűek. Ezeket az adatokat JSON fájlba menti el a program, és ebből a fájlból a következő betöltésnél ki is olvassa.

Az egész játéknak kell, hogy legyen egy teljes grafikus kezelőfelülete. Kell egy fő menü, ahol a játékos betöltheti az előzőleg elmentett adatait, illetve megváltoztathatja a játék alap beállításait. Ezen belül a betöltés után kell egy menü, ahol a játékos indíthat egy kört, vehet kártyákat és összerakhatja a pakliját. A menünek könnyen átláthatónak kell lenni. A kör alatt kell egy grafikus rész ahol látszanak az aktuálisan rendelkezésre álló kártyák, a hátralévő ellenségek, illetve a játék sebessége gombok mellett, amelyekkel lehet azt befolyásolni. A kör végén kell egy felület, amelyen ki van értékelve a kör, illetve fel van tüntetve, hogy a játékos nyert, vagy veszített. Végül, de nem utolsó sorban, kell egy menü amely akkor jelenik meg, ha a játékos megállítja a játékot. Ebben a menüben kellenek gombok a játék folytatására és a kilépésre is.

3 A FELADAT KIDOLGOZÁSÁNAK MENETE

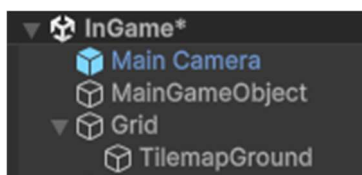
3.1 Fejlesztési környezet kiválasztása

Legelőször a projekthez használt fejlesztői felület kiválasztására került sor. A Unity játékmotor jó lehetőségeket kínál 2D játékok fejlesztéséhez, és egészében teljes mértékben megfelelt annak a célnak amit kitűztünk magunk elé. A beépített rácsrendszer lehetővé teszi, hogy a térképet generáló algoritmus a leghatékonyabb legyen, illetve, ez a rendszer maga kezeli a már elkészített játéktér valós megjelenítését előre megírt szabályok alapján. A Unity UI Toolkit pedig egy stabil, személyre szabható fejlesztői felület létrehozását biztosítja.

A projekt létrehozásakor a legújabb LTS (Long Term Support) verzió, azaz hosszú távú támogatással rendelkező verzió, a 6000.0.49f1 volt, ezért ez mellett döntöttünk. A projektet magát a Unity beépített „Universal 2D” sablonjából hoztuk létre. Ez tartalmazza az alapértelmezett beállításokat 2D jelenetek és 2D játékok futtatásához, illetve a renderelés a Unity Universal Render Pipeline renderelési folyamatláncot, amely minden aspektusában megfelel a projekt követelményeinek.

Az alap sablon jelenetet, amelyet a Unity a projekt létrehozásakor felállít átneveztük „InGame”-re ugyanis ez a jelenet lesz az a jelenet ahol a játék játékmeneti része fog zajlani. Ennek megfelelően létrehoztunk egy játékobjektumot (Game Object) „Grid” névvel. Ez kapott egy Grid komponenst. Ez után a Grid objektumnak alárendelve létrehoztunk még egy játékobjektumot „TilemapGround” névvel. Ez az objektum kapott egy Tilemap és egy Tilemap Renderer komponenst. A létrehozott komponensek mindegyike az alapértelmezett beállításokat használja, ugyanis ezek megfelelnek a projekt céljának. Ezen kívül létrehoztunk még egy játékobjektumot „MainGameObject” névvel. Erre az objektumra fognak később kerülni a játékmenetet irányító logikát tartalmazó szkriptek, illetve a játéktér létrehozásáért felelős algoritmus. Mindezek után az alapértelmezett kamera játékobjektumot állítottuk be. Az objektum Camera komponensében a Size, azaz méret, értékét rövid kísérletezés után beállítottuk 10-re. Ezen az értéken a kamera látoszögének egyik szélén sincsenek csupán részben látszódo négyzetei az előbb létrehozott négyzethálónak, illetve a látható négyzetek száma is megfelel egy nem túl nagy, de egyben nem túl kicsi, játéktér létrehozásához. Végül ezt a játékobjektumot elmentettük egy sablonként (Prefab). Ez lehetővé teszi, hogy a játék összes jelenetében fel tudjuk használni ugyan azt a játékobjektumot úgy, hogy ha megváltoztatjuk a beállításokat, akkor az a sablon minden példányát (Instance) frissíti.

A jelenet végső hierarchiája a következő képpen nézett ki:



ábra 1: A jelenet végső hierarchiája

3.2 Elméleti háttér és műfaj

Az elméleti magvat két, hagyományosan elkülönülő videojáték-műfaj, a tower defense és a kártyajátékok mechanikájának szerves ötvözése adta. Bár mindkét műfaj önállóan is rendkívül népszerű, a kettő szimbiózisa még viszonylag ritka, ami lehetőséget teremtett egyedi, innovatív játékelmény kialakítására.

A tower defense játékok elmélete a taktikus elhelyezkedésen és a hosszú távú tervezésen alapul. Ebben a műfajban a játékosnak nem csupán ügyességre van szüksége, hanem komplex logikai gondolkodásra és előrelátásra, hogy a védelmi vonalakat a leghatékonyabb módon építse ki az ellenséges hullámok megállítása érdekében. A feladat nehézségét a játéktér korlátozott erőforrásai és a rögzített útvonalak mentén hullámokban érkező veszély közötti egyensúly megtalálása jelenti.

Ezzel szemben a kártyajáték-mechanika beillesztése a rendszerbe a rövid távú döntéshozatalt erősíti. Itt a hangsúly a pakli összeállításán és a véletlenszerűen kézbe kerülő lapok megfelelő pillanatban történő kijátszásán van. Ez a megközelítés biztosította, hogy a játékos ne csupán egy előre meghatározott stratégiát kövessen, hanem folyamatosan alkalmazkodjon a rendelkezésre álló lehetőségekhez, kialakítva ezzel saját, egyéni játéktílusát.

A műfaji egyeztetés lényege, hogy a torony védelmi játékok kiszámíthatóbb, tervezést igénylő játékmenetét a kártyák bevonásával változatosabbá tettük. A legnagyobb kihívás e két rendszer olyan módon történő összehangolása volt, hogy azok ne ütközzenek egymással, és egyik se szoruljon háttérbe a másikkal szemben.

- **Stratégiai mélység:** Minden egyes kártyát lehet fejleszteni magasab szintekre (ez növeli erejüket), illetve lehet belőlük több darabot venni. A játékos ezek révén hosszú távú célokat követ, miközben a torony védelmi fázisban taktikai döntéseket hoz.

- **Újrajátszhatóság:** A játékos maga döntheti el, hogy melyik kártyából es mennyit szeretne a paklijába helyezni (bizonyos keretek között, mivel a paklinak van egy maximális nagysága és egy kártyából csakis annyit lehet bele helyezni, amennyi a játékos birtokában van). Mivel kiléenc féle kártya van és 18 a pakli maximális kapacitása, a játékos összesen 198 359 290 368 módon rakhatja össze a pakliját. A kártyarendszer véletlenszerűsége (a pakliból való húzás) és a procedurálisan generált elemek biztosítják, hogy a játék ne váljon unalmassá vagy túlzottan kiszámíthatóvá.
- **Döntési szabadság:** A kártyák kombinációi és a pakli összetételének meghatározása teljes mértékben a játékos kezébe adja az irányítást, ami növeli a játékos motivációját és a fejlődés érzetét.

Az alap ötlet elkészítése után megterveztük a konkrét játékmenetet és a játékban használt kártyákat.

3.3 A játéktér véletlenszerű létrehozása

A stratégiai játékok fejlesztése során az egyik legjelentősebb elméleti és gyakorlati kihívás az újrajátszhatóság biztosítása. Egy statikus pályákkal és mechanikákkal rendelkező játék egy idő után kiszámíthatóvá és unalmassá válik a felhasználó számára, mivel a játékos hamar kiismeri az optimális útvonalakat és védelmi pontokat. A projekt egyik központi kérdése, hogy miként tartható fenn a stratégiai mélység és a folyamatos kihívás egy elméletileg végtelen játékmenet során.

A fenti probléma kiküszöbölésére a feladat kidolgozása során egy automatikus generáló rendszert alkalmaztunk, amely egy 2D grid (rács) rendszerre épül. Ez a megoldás elméleti szinten biztosítja a játéktér véletlenszerűségét, ami az újrajátszhatóság alapja. A rács alapú felosztás lehetővé teszi, hogy a szoftver minden egyes körben más és más, de mégis bizonyos kereteken belül lévő, elrendezést hozzon létre, elkerülve az ismétlődő helyzeteket.

A játéktérület nem statikus, hanem a következő elemei változnak minden indításkor, és az algoritmusba épített önellenőrző funkciók gondoskodnak róla, hogy a generált játéktéren a játék játszható legyen:

- **Az ellenség útvonala:** Az algoritmus minden körben más utat határoz meg, amelyen az ellenségek megpróbálnak eljutni a bázisig.

- **Kiindulópont és bázis:** Az ellenségek kiinduló-helye és a játékos bázisának pozíciója körönként változik, így a játékos nem hagyatkozhat a korábbi sémákra.

Mivel az útvonal változik, a kártyák optimális elhelyezése is minden alkalommal új taktikai elemzést igényel a játékostól.

A 2D grid rendszer és a véletlenszerű elemek beépítése azt eredményezi, hogy a játékosnak folyamatosan alkalmazkodnia kell az aktuális helyzethez. Ez a megközelítés kihangsúlyozza a játékos által meghozott döntéseket és azok fontosságát, mert a változó környezetben különböző kártyakombinációk válhatnak sikeressé. A cél egy olyan rendszer kialakítása, amely mindig új kihívásokat kínál, de közben nem válik túl bonyolulttá vagy kezelhetetlenné.

Először létrehoztuk a „MapHandler.cs” szkriptet és hozzárendeltük a MainGameObject játékbjektumhoz. A pálya generálásának alapját egy kétdimenziós rács jelenti, amelynek méretét a `mapSize` változó határozza meg. Ez a változó `Vector2Int` típusú, amely a pálya szélességét és magasságát tartalmazza. A pálya vizuális megjelenítését a Unity Tilemap rendszere biztosítja. A tilemap minden mezője egy adott típusú csempét tartalmazhat, üres mezőt, ellenség kiindulási pontot, bázist vagy az útvonal részét képező mezőt. A különböző csempékhez külön `Tile` illetve `RuleTile` objektumok tartoznak, amelyek a grafikus megjelenítést biztosítják.

A pálya létrehozása a `createMap` metódus meghívásával történik. A folyamat első lépése a pálya teljes területének inicializálása. A program két egymásba ágyazott ciklus segítségével végigiterál a pálya minden mezőjén, és minden koordinátára elhelyez egy alapértelmezett üres csempét. Ezzel biztosíthattuk, hogy a pálya minden pontja definiált állapotban legyen a generálás további lépései előtt. A pálya inicializálása után a rendszer kiválasztja a játékos bázisának helyét. Ezt a `selectBaseTile` metódus végzi, amely véletlenszerű koordinátát generál a pálya határain belül. Az így kapott koordináta lesz a bázis pozíciója.

A program ezt követően meghatározza, hogy a pálya melyik negyedében helyezkedik el a kiválasztott mező. Ez a `getMapQuarter` metódus segítségével történik, amely a pályát négy részre osztja: jobb felső, bal felső, bal alsó és jobb alsó negyedre. A rendszer ennek az információnak a segítségével tudja eldönteni, hogy az ellenségek honnan induljanak.

Az ellenségek kiindulási pontjának kiválasztását a `selectEnemyTile` metódus végzi. A rendszer úgy működik, hogy az ellenségek mindig a pálya egy olyan szélén jelennek meg, amely a bázistól távol esik. A kiválasztott negyedről függően a program a pálya egy adott

élén generál egy véletlenszerű koordinátát. Például ha a bázis a jobb felső negyedben található, akkor az ellenségek a pálya bal alsó negyedének a bal oldalán vagy alsó szélén jelenhetnek meg. Ezzel a megoldással biztosítottuk, hogy a két fontos pont ne legyen egymáshoz túl közel, így a játékosnak legyen ideje reagálni az érkező ellenségekre.

Miután a bázis és az ellenség kiindulási pontja meghatározásra került, a rendszer ezeket a pozíciókat eltárolja és megjeleníti a pályán. A tilemap megfelelő mezőire elhelyezésre kerülnek a bázis és az ellenség spawn pontját jelző csempék. Ezzel párhuzamosan a rendszer egy listában eltárolja azokat a pályanegyedeket, amelyek már foglaltak. Ez az információ később fontos szerepet játszik az útvonal generálásánál.

Az útvonal generálása a `generatePath` metódus feladata. Ennek a folyamatnak a célja egy olyan összefüggő út létrehozása, amely az ellenségek kiindulási pontjától a játékos bázisáig vezet. Az algoritmus több köztes pontot, úgynevezett waypointokat használ, amelyek segítségével az útvonal változatosabb és kevésbé egyenes lesz. A waypointok létrehozását a `createPathWaypoints` metódus végzi. A waypoint generálás során a rendszer megvizsgálja a pálya négy negyedét, és minden olyan negyedben elhelyez egy waypointot, amelyet még nem foglal el sem a bázis, sem az ellenségek kiindulási pontja. A waypoint pontos koordinátája az adott negyed határain belül véletlenszerűen kerül kiválasztásra. Ennek köszönhetően minden pálya más és más útvonallal rendelkezik.

Miután a waypointok listája elkészült, a program sorban összeköti ezeket a pontokat. Az összekötést a `createPath` metódus végzi. Ez az algoritmus lépésről lépésre halad a kiindulási ponttól a célpont felé. A program minden lépésben meghatározza azokat az irányokat, amelyek közelebb viszik a jelenlegi pozíciót a célhoz. Ezeket az irányokat preferált irányokként elmenti. A rendszer megvizsgálja a preferált irányokban szomszédos mezőket, és kiválasztja azokat, amelyek szabadok és megfelelnek bizonyos feltételeknek. Ha több megfelelő irány is létezik, a program véletlenszerűen választ közülük, így az útvonal minden generálás során eltérő lehet. Ha egyik preferált irány sem használható, a program más szomszédos mezőket is megvizsgál, hogy elkerülje a zsákutcák kialakulását.

A szomszédos mezők ellenőrzését a `checkFreeDir` metódus végzi. Ez a függvény megvizsgálja, hogy az adott mező a pálya határain belül van-e, még nem része-e az útvonálnak, valamint rendelkezik-e elegendő szabad szomszéddal ahhoz, hogy a későbbiekben az út folytatható legyen. Az utóbbi feltételt a `tileHasEscape` metódus ellenőrzi. Ez a függvény megszámlolja, hogy az adott mező hány irányban rendelkezik szabad kijáráttal. Amennyiben túl kevés lehetőség áll rendelkezésre, a mező nem kerül felhasználásra, mivel nagy valószínűséggel zsákutcát eredményezne.

A rendszer minden lépésben hozzáadja az aktuális mezőt az útvonal listájához. Ez a folyamat addig folytatódik, amíg az algoritmus el nem éri a következő waypointot vagy a bázis pozícióját. A teljes útvonal tehát több kisebb szakaszból áll össze, amelyek a waypointok között jönnek létre. Miután az útvonal generálása befejeződött, a program végigiterál a létrehozott koordináták listáján, és minden mezőn elhelyez egy útvonalat jelző csempét a tilemapen. Ez teszi láthatóvá a játékos számára az ellenségek által követett útvonalat.

A pálya generálásának utolsó lépéseként a rendszer létrehozza a szükséges játékobjektumokat is.

A játék egyik fontos mechanikáját az építmények és varázslatok kezelése jelenti, amelyet a „BuildingHandler.cs” nevű szkriptben valósítottunk meg. Ez a komponens felelős a játékos által kiválasztott kártyák feldolgozásáért, az épületek elhelyezéséért a pályán, valamint a különböző varázslatok alkalmazásáért.

A rendszer működéséhez két másik fontos komponensre támaszkodik: a MainGameLogic és a MapHandler szkriptekre. A Start metódus során a program lekéri ezeknek a komponenseknek a referenciáját az aktuális GameObjectról, majd eltárolja azokat. A pályakezelő szkriptből a program az ellenségek útvonalát is lekéri, amely egy lista formájában tartalmazza az útvonalat alkotó csempék koordinátáit. Ez az információ kulcsfontosságú az építmények elhelyezésének ellenőrzése során.

A rendszer különbséget tesz többféle építmény- és kártyatípus között. Ezek a típusok a MainGameLogic.CardTypes enumerációban vannak meghatározva. A program külön listákban tárolja azokat a típusokat, amelyek épületnek, csapdának vagy varázslatnak számítanak.

A pályán már elhelyezett objektumok nyilvántartására a rendszer egy OccupiedPlot nevű struktúrát használ. Ez a struktúra három adatot tartalmaz: az adott mező koordinátáját, az ott található objektum típusát, valamint az objektumhoz tartozó GameObject referenciát. Az összes elfoglalt mező egy occupiedTiles nevű listában kerül eltárolásra, amely lehetővé teszi a gyors ellenőrzést az építés során.

Amikor a játékos kiválaszt egy kártyát a felhasználói felületen, a cardSelected metódus kerül meghívásra. Ez a függvény elindítja az építés folyamatát a beginBuilding metódus segítségével. A folyamat során a rendszer eltárolja a kiválasztott kártya típusát, majd inicializálja az ehhez tartozó objektumokat.

Az építés egyik fontos eleme az úgynevezett blueprint rendszer. A blueprint egy vizuális

előnézet, amely megmutatja a játékosnak, hogy az adott objektum elhelyezhető-e a kiválasztott mezőn. A rendszer minden építménytípushoz két blueprint változatot használ: egy érvényes és egy érvénytelen állapotot jelző verziót. Ezek az objektumok a pályán követik az egér pozícióját, és attól függően jelennek meg, hogy az adott hely alkalmas-e az építésre.

Az építés megkezdésekor a program példányosítja a megfelelő blueprint objektumokat, majd eltárolja azokat egy szótárban. Ez a szótár kulcsként egy `BuildingAssetType` enumerációt használ, amely három lehetséges típust különböztet meg: maga az épület objektuma, az érvényes blueprint és az érvénytelen blueprint.

Az építés során a rendszer folyamatosan figyeli az egér pozícióját. Ez a folyamat az `Update` metódusban történik, ahol a program minden képkockában meghívja az `updateBlueprint` függvényt, amennyiben a játékos éppen építési módban van. A függvény kiszámítja az egér aktuális koordinátáját a rácsrendszer koordinátáira kerekítve, majd ennek megfelelően mozgatja a blueprint objektumokat.

Az elhelyezés érvényességének ellenőrzése a kiválasztott kártya típusától függ. Tornyok esetében a rendszer ellenőrzi, hogy az adott mező nem része-e az ellenségek útvonalának, valamint hogy nincs-e már ott másik építmény. Csapdák esetében ennek az ellenkezője történik: ezek csak az útvonalon helyezhetők el. A varázslatok pedig nem önálló objektumok, hanem már meglévő építményekre alkalmazható hatások, ezért ezeknél a rendszer azt vizsgálja, hogy a célnégyzeten megfelelő típusú objektum található-e.

Amennyiben a feltételek teljesülnek, a program az érvényes blueprintet jeleníti meg, ellenkező esetben pedig az érvénytelen változatot.

A tényleges építés a `placeNewBuilding` metódus meghívásával történik. Ez a függvény csak akkor hajtodik végre, ha az aktuális pozíció érvényes, és a játékos valóban építési módban van. Amennyiben a kiválasztott kártya egy építmény, a rendszer létrehozza a megfelelő `GameObject` példányt a pályán. A létrehozott objektum koordinátája a `tilemap` rácsához igazított pozíció lesz. Az új objektum ezután bekerül az `occupiedTiles` listába, így a rendszer a későbbiekben figyelembe tudja venni az elhelyezési ellenőrzések során.

Ha a kiválasztott kártya egy varázslat, akkor a folyamat eltérően működik. A varázslatok nem hoznak létre új objektumot. A rendszer a kiválasztott mező alapján meghatározza a célobjektumot, majd meghívja annak megfelelő komponensét. A különböző épülettípusok szkriptjei saját `applyEffect` metódussal rendelkeznek, amely a varázslat paraméterei alapján módosítja az adott objektum tulajdonságait.

A varázslatok adatait egy `SpellScriptableObject` objektumban tároljuk. Ez az objektum

tartalmazza a hatás erősségét és időtartamát. Amikor a varázslat alkalmazásra kerül, ezek az értékek kerülnek átadásra az adott építménynek. Egy speciális eset a bázis gyógyítása, amely közvetlenül a játékos életerejét módosítja a MainGameLogic komponensben.

Az építési folyamat befejezése a finishBuilding metódus segítségével történik. Ez a függvény eltávolítja a blueprint objektumokat, visszaállítja az ideiglenes változókat alapállapotba, és kilép az építési módból. Ezzel a rendszer készen áll arra, hogy a játékos újabb kártyát válasszon ki és új építményt helyezzen el a pályán.

3.4 Adatkezelés és optimalizálás

A projekt a JSON (JavaScript Object Notation) formátumot használja az adatok tartós tárolására. A JSON választása mellett több szakmai érv szól:

- **Könnyű szerkeszthetőség:** A fejlesztési fázisban a kártyák értékei egyszerű szövegfájlként módosíthatók anélkül, hogy a programkódot újra kellene fordítani. Ez sok időt is megspórol és általánosan jó, ha az adatok nem egyenesen a kódban vannak tárolva, hanem külön erre a célra kialakított fájlokban..
- **Strukturáltság:** Ez a formátum lehetővé teszi az adatok hierarchiában való elrendezését, ami ideális a kártyák tulajdonságainak és a mentett játék állapotok rögzítésére. A json alapja, hogy az adatokat objektumokban tárolja. Ez ideális, mivel mindegyik kártyához több adat tartozik. Ez a formátum lehetővé teszi, hogy ezeket az adatokat rendszerezetten és hierarchiába rendezve legyenek tárolva.
- **Hatékonyág:** A program a betöltéskor gyorsan beolvassa a fájlt, és a benne lévő információkat a játék aktuális állapota és jelenlegi szükséglete alapján szűri ki. Nem kell minden adatot a kódon belül változóban tárolni. A json fájl..

A memória-takarékosság és az egyszerűség érdekében a projekt egy köztes adatkezelő réteget alkalmaz. A JSON fájlokból beolvasott információk nem közvetlenül a játékobjektumokba töltődnek be minden alkalommal, hanem úgynevezett Scriptable Object struktúrákba kerülnek. Ez a technika azért jelentős, mert az adatok teljes mértékben elkülönülnek a logikától, így a kód tisztább és könnyebben karbantartható marad. Illetve ugyan azt az adatot több helyen is fel lehet használni, de nem kell az értéket külön-külön tárolni, elég csak a referenciát tárolni a Scriptable Object változójáról. Így elkerülhető, hogy az azonos kártyatípusok és játékobjektumok feleslegesen duplikálják az adataikat a memóriában.

Elsőként létrehoztuk az összes szükséges JSON fájlt. A játékos 4 különböző állapotot menthet el egyszerre külön-külön profilként. Mindegyik profilt bármikor vissza lehet állítani alapértelmezett állapotába. Elsőként létrehoztuk a négy profilnak megfelelő négy üres JSON fájlt. Ezután létrehoztunk egy csontváz profil fájlt (Skeleton Profile). Ez egy sablonként működik egy profil visszaállításakor. Ez után létrehoztunk egy JSON fájlt mindegyik kártyának és feltöltöttük őket a megfelelő adatokkal, megfelelően struktúrálva.

Létrehoztunk Scriptable Object szkripteket amelyek ezeket a JSON fájlokat reprezentálják. 3 különböző ilyen szkriptelhető objektum sablon készült: egy a torony adatainak, egy az összes féle csapda adatainak, és egy az összes féle effekt adatainak. Ha egy adott kártyának nincs az adott értéke (mint például az alap csapdának nincs effekt hossz vagy effekt erőssége értéke), az a JSON fájlban nincs is reprezentálva. Ezt azért csináltuk, mert a JSON deszerializáló, amelyet használni fogunk, automatikusan kezeli az ilyen null értékeket. Ha az osztályban amelyre deszerializálja a fájlt, van olyan mező amely nem létezik a fájlban, automatikusan null értéket rendel hozzá. Ezen kívül létrehoztunk egy sablont a kártyák textúráinak és az ellenség adatainak tárolására. A játékos aktuálisan betöltött profiljának az adatainak a tárolására is szkriptelhető objektumot használunk, így erre is létrehoztunk egy sablont.

```
using System.Collections.Generic;
using UnityEngine;

[CreateAssetMenu(fileName = "CardTexturesScriptableObject", menuName = "Scriptable Objects/CardTexturesScriptableObject")]
2 references | ☑ Unity Script
public class CardTexturesScriptableObject : ScriptableObject
{
    3 references | ☑ Unity Serialized Field
    public Dictionary<MainGameLogic.CardTypes, Texture2D> textures;
}
```

ábra 2: CardTextureScriptableObject szkript implementációja

```
using UnityEngine;

[CreateAssetMenu(fileName = "EnemyScriptableObject", menuName = "Scriptable Objects/EnemyScriptableObject")]
2 references | ☑ Unity Script
public class EnemyScriptableObject : ScriptableObject
{
    3 references | ☑ Unity Serialized Field
    public int speed;
    2 references | ☑ Unity Serialized Field
    public int health;
    3 references | ☑ Unity Serialized Field
    public int weakness;
}
```

ábra 3: EnemyScriptableObject szkript implementációja

```

using System.Collections.Generic;
using UnityEngine;

[CreateAssetMenu(fileName = "PlayerProfileScriptableObject", menuName = "Scriptable Objects/PlayerProfileScriptableObject")]
3 references | ☑ Unity Script
public class PlayerProfileScriptableObject : ScriptableObject
{
    13 references | ☑ Unity Serialized Field
    public int gold;
    17 references | ☑ Unity Serialized Field
    public Dictionary<MainGameLogic.CardTypes, SaveLoadSystem.Card> cards;
}

```

ábra 4: PlayerProfileScriptableObject szkript implementációja

A kártyákhoz tartozó sablonokban létrehoztunk egy saját „Init” metódust, amely paraméterként megkapja az objektumot amelybe az adott kártya adatai lettek deszerializálva és a kártya a már korábban betöltött profilban elmentett szintjét. Scriptable Object szkriptnek, bár egy osztály, nem lehet konstruktor metódusa. Ezért kellett létrehoznunk egy saját inicializáló metódust, amelyet az objektum létrehozása után manuálisan hívunk meg. A kártyák objektumainál továbbá, mivel az összes féle csapda illetve effekt kártyát csak egy típussablon reprezentálja (TrapScriptableObject és SpellScriptableObject), fontos hogy a helyes példányon állítsuk be a mezőket. Ezért használjuk a `this` kulcsszót a változók beállításánál.

```

using System.Collections.Generic;
using UnityEngine;

[CreateAssetMenu(fileName = "SpellScriptableObject", menuName = "Scriptable Objects/SpellScriptableObject")]
13 references | ☑ Unity Script
public class SpellScriptableObject : ScriptableObject
{
    2 references | ☑ Unity Serialized Field
    public Dictionary<string, string> title;
    2 references | ☑ Unity Serialized Field
    public Dictionary<string, string> description;
    4 references | ☑ Unity Serialized Field
    public Dictionary<string, int> costs;
    9 references | ☑ Unity Serialized Field
    public float effectstrength;
    6 references | ☑ Unity Serialized Field
    public float effectduration;

    4 references
    public void Init(SaveLoadSystem.SpellData data, int level)
    {
        SaveLoadSystem.SpellStats stats = data.stats[$"level{level}"];

        this.title = data.title;
        this.description = data.description;
        this.costs = data.costs;
        this.effectstrength = stats.effectstrength;
        this.effectduration = stats.effectduration;
    }
}

```

ábra 5: SpellScriptableObject szkript implementációja (példa)

Ezután létrehoztuk a „SaveLoadSystem.cs” szkriptet, amely az adatok betöltéséért, elmentéséért, a fájlok kezeléséért és a szkriptelhető objektumok beállításáért felelős. A rendszerben úgynevezett Singleton mintát alkalmaztunk, amellyel biztosítottuk, hogy a

program futása során csak egyetlen példánya létezzen. Ez a megoldás azért fontos, mert a mentési és betöltési folyamatoknak globálisan elérhetőnek kell lenniük a játék különböző részei számára. A szkript létrehoz egy statikus Instance változót, amely a jelenleg aktív példányra mutat. Amennyiben a program egy új példányt próbálna létrehozni, az előzőhöz képest felesleges objektum automatikusan megsemmisül.

A szkript a Newtonsoft.Json könyvtár segítségével végzi az adatok szerializálását és deszerializálását. A szerializáció során a program a memóriában lévő objektumokat JSON formátumba alakítja és fájlba írja. A deszerializáció során ennek az ellenkezője történik: a JSON fájl tartalma visszaalakításra kerül programon belüli objektumokká. [7]

A mentési rendszer a játékos adatait külön profilokban tárolja. A rendszer négy különálló mentési helyet biztosít, amelyeket a program egy enumeráció segítségével azonosít. Ezek a profilok külön JSON fájlokban kerülnek tárolásra a rendszer által meghatározott mentési könyvtárban. A könyvtár elérési útját a program a Unity által biztosított Application.persistentDataPath változóból olvassa ki, amely minden platformon egy biztonságos, alkalmazás-specifikus tárolási helyet biztosít.

Amikor a játék először indul el, a rendszer ellenőrzi, hogy létezik-e már a mentési könyvtár. Ha ez a könyvtár még nem létezik, akkor a program automatikusan létrehozza azt, majd minden profil számára elkészít egy alapértelmezett mentési fájlt a SkeletonProfile JSON fájl alapján.

A mentési rendszer a játékos aranyának mennyiségét, és az összes kártyához tartozó adatot is eltárolja. Minden kártya rendelkezik három fontos tulajdonsággal: a szintjével, a pakliban lévő darabszámával, valamint azzal, hogy a játékos hány példányt birtokol belőle. Ezek az adatok egy külön Card nevű adatstruktúrában kerülnek tárolásra. A játékos teljes profilja pedig egy Profile osztályban található, amely tartalmazza az arany mennyiségét és az összes kártya adatait.

A JSON fájlok szerkezetének egyszerűsítése érdekében a program egy köztes adatstruktúrát is használ, amelynek neve ProfileRaw. Ez a struktúra közvetlenül a JSON fájl felépítését tükrözi, ahol a kártyák azonosítói szöveggént szerepelnek. A betöltés során a program ezeket a szöveges kulcsokat egy enumeráció típusba konvertálja. Ez a lépés azért szükséges, mert a program további részei már típusbiztos módon, enumerációként hivatkoznak a különböző kártyákra.

Amikor a játékos kiválaszt egy mentési profilt, a loadProfile metódus kerül meghívásra. Ez a metódus megkeresi a megfelelő JSON fájlt, majd annak tartalmát beolvassa és deszerializálja. Az így kapott adatokat a program egy Profile objektumba tölti, majd

továbbadja a játék többi részének. A játék aktuális állapotának kezelésére egy `PlayerProfileScriptableObject` használtunk fel, amely a memóriában tartalmazza a játékos aktuális adatait. A betöltött profil adatai ebbe az objektumba kerülnek másolásra.

A betöltési folyamat következő lépése a játék kártyáinak és egyéb objektumainak inicializálása. Ehhez a program külön JSON fájlokat használ minden egyes kártyatípushoz. Ezek a fájlok tartalmazzák a kártyák nevét, leírását, költségét és különböző szintekhez tartozó statisztikáit. A `updateScriptableObjectData` nevű metódus feladata ezeknek az adatoknak a feldolgozása. A metódus betölti a megfelelő JSON fájlokat a Unity Resources könyvtárból, majd azokat a megfelelő adatstruktúrákba deszerializálja. Miután az adatok beolvasásra kerültek, a program meghatározza az adott kártya aktuális szintjét a játékos profiljából. Ez alapján kiválasztja a megfelelő statisztikákat, majd meghívja az adott `ScriptableObject` `Init` metódusát. A rendszer ezen kívül az ellenségek adatait is betölti egy külön JSON fájlból. Az ellenség sebessége, életeréje és gyengesége egy `EnemyData` nevű struktúrába kerül, majd ezek az értékek átadásra kerülnek az `EnemyScriptableObject` objektumnak.

A mentési rendszer további feladata a kártyákhoz tartozó grafikus elemek betöltése is. A `loadCardTextures` metódus a kártyák képeit tölti be a Resources mappából. Ezek az adatok egy szótár struktúrában kerülnek eltárolásra, ahol a kulcs a kártya típusa, az érték pedig a hozzá tartozó textúra. A textúrák ezután egy `CardTexturesScriptableObject` objektumba kerülnek, amely lehetővé teszi, hogy a grafikus felület könnyen hozzáférjen a megfelelő képekhez.

A játék során a játékos vásárolhat kártyákat, fejlesztheti azokat, illetve hozzáadhatja vagy eltávolíthatja őket a paklijából. Ezeket a műveleteket külön metódusok kezelik, mint a `buyCard`, `upgradeCard`, `addCardToDeck` és `removeCardFromDeck`. Mindegyik metódus módosítja a játékos profiljában tárolt adatokat, majd újra meghívja az adatfrissítő metódust, hogy a változások azonnal érvénybe lépjenek. Amikor a játékos elmenti a játék állapotát, a `saveProfile` metódus kerül végrehajtásra. Ez a metódus létrehoz egy új `Profile` objektumot a jelenlegi játékállapot alapján, majd azt JSON formátumba szerializálja. Az így létrehozott szöveges adat a megfelelő profil fájlba kerül kiírásra.

3.5 Felhasználói felület (GUI)

A kezelőfelületnek könnyen átláthatónak kell lennie. Ez azt jelenti, hogy a játékosnak minimális terhelés mellett kell hozzájutnia a fontos információkhoz. Egy stratégiai játéknál ez az alábbi részek elkülönítését követeli meg:

1. **Főmenü:** Ez a rész felelős a játékos és a rendszer közötti alapvető interakciókért, mint például a korábban mentett adatok betöltése vagy a játék alapvető beállításainak módosítása.
2. **Előkészületi felület:** A játékmenet elméleti része a felkészülés, ahol a játékos a boltban kártyákat vásárolhat és összeállíthatja a pakliját.
3. **Harci felület (In-game UI):** A tényleges játékmenet alatt a GUI-nak folyamatos visszajelzést kell adnia. Kötelezően meg kell jelenniük a következő elemeknek:
 - Az aktuálisan rendelkezésre álló kártyák megjelenítése.
 - A hátralévő ellenségek száma.
 - Vezérlőgombok, amelyekkel a játék sebessége befolyásolható.

A felhasználói interakcióhoz hozzátartozik a játék adott állapotainak egyértelmű jelzése. A projekt részeként olyan felületeket dolgoztunk ki, amelyek kezelik a játékmenet megszakítását és lezárását:

- **Szünet menü:** Biztosítja a játék folyamatosságának irányítását(folytatás vagy kilépés gombokkal).
- **Értékelő felület:** A kör végén megjelenő felület, itt történik a győzelem vagy vereség kiértékelése és a kör eredményeinek prezentálása.

A GUI tervezésének célja, hogy a technikai háttérfolyamatok, mint a kártyák Scriptable Object-ekből való beolvasása vagy a 2D grid alapú pályagenerálás, a játékos számára érthető és kezelhető vizuális információkká alakuljanak. Első lépésben létre hoztunk három UXML fájlt. Egyet a fő menünek, egyet a harci felületnek és egyet a fő menüben a kártyákat megjelenítő ablakban egy sor sablonjának. Mindegyik fájlhoz létrehoztunk egy-egy USS fájlt is, amelyben a stílus beállításai vannak tárolva. Mindezek után létrehoztunk az összes szükséges textúrát es menüt.

A „MainMenuHandler.cs“ nevű szkriptben a játék főmenüjének működését valósítottuk meg. A program feladata a felhasználói felület kezelése, a profilok kiválasztása és törlése,

valamint a kártyarendszerhez tartozó menü működtetése. A szkriptet a Unity motor UI Toolkit rendszerére építettük fel, és a játék főmenüjének logikai vezérlését látja el. A kód kapcsolatban áll a SceneHandler és a SaveLoadSystem osztályokkal, amelyekkel a jelenetek közötti váltást, illetve a játékos profiljának betöltését és mentését kezeltük. Az osztályban számos szótár típusú adatszerkezetet használtunk a különböző gombok rendszerezésére. Ezek a szótárak az egyes menüpontokhoz tartozó gombokat tárolják.

A program több VisualElement, Label, valamint ListView típusú változót is használ. Ezek a UI Toolkit felület különböző elemeit reprezentálják. A kártyák adatainak tárolására a programban több belső osztályt definiáltunk. A CardData osztály egy adott kártya minden fontos adatát tartalmazza. A CardStats osztály az adott kártya a betöltött profilhoz kapcsolódó statisztikáit tárolja. A kártyák alap statisztikáit a BaseCardStats osztály reprezentálja, amely tartalmazza a vásárlási és fejlesztési költségeket. Ebből származik a TowerData, TrapData és SpellData osztály, amelyek a különböző kártyatípusok specifikus statisztikáit tartalmazzák.

A program működésének kezdeténél kerül inicializálásra a jelenetkezelő és a mentési rendszer referenciája. Ezután a program lekéri a UIDocument komponensből a gyökér vizuális elemet, majd ezen keresztül eléri a felhasználói felület egyes részeit, a főmenüt, a profilválasztót vagy a kártyamenüt. A metódus beállítja a különböző menük kezdeti láthatóságát attól függően, hogy van-e már betöltött játékosprofil. Amennyiben nincs betöltött profil, a profilválasztó és a profil törlésére szolgáló menü jelenik meg. Ha viszont már van aktív profil, a program automatikusan megnyitja az előcsata menüt, és betölti az adott profil adatait.

A Start metódus ezen kívül meghívja azokat a függvényeket, amelyek beállítják a gombokat és azok eseménykezelőit. A createCardDataList metódus feladata a kártyák adatainak összegyűjtése és egy listába rendezése. A függvény végigiterál a játékban létező összes kártyatípuson, majd a kártya típusától függően lekéri a megfelelő ScriptableObject adatait. Ezekből létrehoz egy CardData objektumot, amely tartalmazza a kártya megjelenítéséhez szükséges információkat. A metódus ezen kívül lekéri a játékos profiljában tárolt statisztikákat is.

A bindCardElements metódus felelős azért, hogy a ListView egy adott sorához hozzárendelje a megfelelő kártyaadatokat. A függvény lekéri a vizuális elemeket, a textúra helyét, a statisztikák mezőit és a gombokat. Ezután a kártya típusának megfelelően kitölti a statisztikák értékeit. Például torony esetén a sebzés, hatótáv és támadási sebesség jelenik meg. A metódus ezen kívül beállítja a gombok eseménykezelőit a vásárlás, fejlesztés,

pakliba helyezés vagy pakliból eltávolítás műveletekhez.

A program több különálló metódust tartalmaz a gombok inicializálására és az eseménykezelők hozzárendelésére. A `setProfileButtons`, `setProfileDeleterButtons` és `setPreBattleMenuButtons` metódusok a felhasználói felület megfelelő gombjait töltik fel a korábban definiált szótárakba. A `setProfileButtonEvents`, `setProfileDeleterButtonEvents` és `setPreBattleMenuButtonEvents` metódusok pedig a gombokhoz tartozó eseményeket rendelik hozzá.

A különböző felhasználói műveletekhez eseménykezelő függvények tartoznak. A `OnProfileButtonClicked` metódus betölti a kiválasztott játékosprofil, majd megjeleníti az előcsata menüt. A `OnSaveButtonClicked` elmenti a játékos aktuális profilját, majd visszalép a profilválasztó menübe. A `OnStartButtonClicked` elindítja a játékot az aktuális jelenet lecserélésével.

A profil törlésének kezelése a `OnProfileDeleteButtonClicked`, `OnCancelButtonClicked` és `OnConfirmButtonClicked` metódusokban történik. Amikor a játékos törölni szeretne egy profilt, a program egy megerősítő ablakot jelenít meg. Ha a játékos megerősíti a műveletet, a mentési rendszer törli az adott profil adatait.

A felhasználói felület állapotának kezelésére két rekurzív segédfüggvény is található a programban. A `setInteractableRecursive` függvény egy adott UI elem és annak gyerekeinek interakciós állapotát módosítja, míg a `setVisibleRecursive` a láthatóságukat állítja be. Ezek lehetővé teszik, hogy egy teljes menürész egyszerre legyen engedélyezve vagy elrejtve.

A pakliban található kártyák számának meghatározását a `getCardInDeckCount` metódus végzi. A függvény végigiterál a játékos profiljában tárolt kártyákon, és összeszámolja, hogy hány darab van jelenleg a pakliban. Az `updatePlayerStatLabels` metódus pedig a felhasználói felületen frissíti a pakliban lévő kártyák számát és a játékos aranyának mennyiségét.

Ez után létrehoztuk az „`InBattleMenuHandler.cs`” szkriptet amely a harci felület kezeléséért felelős. A játékos statisztikáinak megjelenítéséhez a program két Label elemet használ: a `healthLabel` és az `energyLabel` mezőket. Ezek a játékos aktuális életerejét és energiáját jelenítik meg a képernyőn. Ezen kívül létezik egy `energyWarningLabel`, amely akkor jelenik meg, ha a játékos nem rendelkezik elegendő energiával egy művelet végrehajtásához.

A kártyák megjelenítésére a program egy szótár típusú adatszerkezetet használ, amely a kártyagombok azonosítóját és a hozzájuk tartozó Button objektumokat tárolja. A szótár öt bejegyzést tartalmaz, amelyek a játékos kezében lévő kártyák helyeit reprezentálják. Ezek

az azonosítók a card0, card1, card2, card3 és card4 kulcsok.

A displayLabels nevű enumeráció két értéket definiál, amelyek a játékos statisztikáinak típusait reprezentálják. Ezek a HEALTH és ENERGY értékek, amelyeket a program a statisztikai címkék frissítéséhez használ.

Az Awake metódus során a program feltölti a kártyagombok szótárát. Egy ciklus segítségével minden kártyahelyhez hozzárendeli a megfelelő Button objektumot a felhasználói felületen. Ezt követően a program létrehozza a cardTextures szótárát, amely a kártyák típusát a hozzájuk tartozó textúrával párosítja. A metódus beállítja a különböző UI elemek kezdeti állapotát is. A szünet jelző címke, a szünet menü, az energiafigyelmeztetés és a csata végi menü kezdetben rejtett állapotban vannak.

Ezen kívül a program eseménykezelőket rendel a Continue, Quit és Finish gombokhoz. A paklikezelő rendszer OnCardDraw eseményére is feliratkozik, amely akkor hívódik meg, amikor a játékos új kártyát húz.

A updateLabel metódus feladata a játékos statisztikáinak frissítése a felhasználói felületen. A függvény egy paraméterként kapott enumeráció alapján dönti el, hogy az életerő vagy az energia címkéjét kell frissíteni. Az aktuális értékeket a MainGameLogic objektumból olvassa ki. A displayEnergyWarning metódus egy Coroutine, amely rövid ideig megjelenít egy figyelmeztetést, ha a játékos nem rendelkezik elegendő energiával az adott kártya kijátszásához. A metódus először láthatóvá teszi a figyelmeztető címkét, majd két másodperc várakozás után ismét elrejtí az.

A csata befejezésének kezelését a displayFinishMenu metódus végzi. A függvény paraméterként megkapja, hogy a játékos győzött-e, valamint hogy mennyi jutalmat kap. A metódus ennek megfelelően beállítja a menü szövegét, például „VICTORY!” vagy „DEFEAT!” felirattal, majd megjeleníti a jutalomként kapott arany mennyiségét. Ezután a csata végi menü láthatóvá válik.

A pakliból húzott kártyák megjelenítését a Deck_OnCardDraw metódus végzi, amely a DeckHandler eseménykezelőjeként működik. Amikor egy új kártya kerül a játékos kezébe, a függvény beállítja a megfelelő gomb háttérképét a kártya textúrájára. Ezen kívül meghatározza a kártya energia költségét a megfelelő ScriptableObject adataiból, majd ezt az értéket a gomb szövegeként jeleníti meg.

A szünet funkció kezelését két metódus valósítja meg. Az OnPauseButtonPressed a játék ideiglenes megállítását kezeli. Amikor a játékos megnyomja a megfelelő bemeneti gombot, a program meghívja a MainGameLogic.togglePause függvényt, amely megállítja vagy folytatja a játékot. A OnPauseMenuButtonPressed metódus hasonló módon működik,

azonban ez a szünet menüt nyitja meg vagy zárja be. A `OnContinueButtonClicked` metódus a szünet menü „Continue” gombjához tartozik. A függvény bezárja a szünet menüt és folytatja a játékot. A `OnQuitButtonClicked` metódus pedig kilép a csatából, és visszalépteti a játékost a főmenübe a `SceneHandler` segítségével.

4 A FELADAT KIDOLGOZÁSÁNAK EREDMÉNYEI

A fejlesztési folyamat végére egy teljes körűen működő, „Kártyavarak” című hibrid stratégiai játékot hoztunk létre, amely sikeresen ötvözi a toronyvédelmi és a kártyajáték-mechanikákat. A projekt egyik legfontosabb eredménye a 2D grid alapú pályageneráló rendszer (MapHandler.cs) stabil működése. Sikerült megvalósítanunk, hogy minden egyes indításkor egyedi útvonal jöjjön létre az ellenségek számára. A BuildingHandler szkript implementálásával létrehoztunk egy interaktív építési rendszert: kilenc különböző kártyatípus (tornyok, csapdák, varázslatok) teljes körű integrációja, működő blueprint rendszer, amely vizuális visszajelzést ad az elhelyezés érvényességéről és a varázslatok sikeresen alkalmazhatók a már meglévő építményekre vagy a bázis gyógyítására, közvetlenül módosítva azok Scriptable Object-ben tárolt értékeit. A SaveLoadSystem segítségével megvalósítottunk egy robusztus, JSON alapú adattárolási struktúrát is. A játékos négy különálló profilban mentheti el a haladását. A kártyák statisztikái (szint, darabszám, költség) külső JSON fájlokból töltődnek be, ami lehetővé teszi a játékegyensúly gyors finomhangolását a kód módosítása nélkül. A memóriakezelést a Scriptable Objectek használata optimalizálja, így az azonos típusú kártyák nem duplikálják az adatokat. A Unity UI Toolkit segítségével egy reszponzív és átlátható kezelőfelületet hoztunk létre. A fő menü és előcsata menü lehetővé teszi a profilkezelést, a kártyák vásárlását, fejlesztését és a pakli összeállítását (deck-building). A harci felület (In-game UI) dinamikus megjeleníti a játékos életerejét, energiáját, a hátralévő ellenségek számát és a kézben lévő kártyákat. Sikerült implementálnunk a szüneteltetési funkciót, a „győzelem/vereség” ablakokat és az energiaköltségre figyelmeztető jelzéseket.

A projektet a Unity 6000.0.49f1 LTS verziójában készítettük, kihasználva a Universal Render Pipeline (URP) előnyeit. A C# nyelv objektumorientált (OOP) sajátosságait követve a kód tiszta, eseményvezérelt és komponens-alapú maradt. A tesztelési fázis igazolta, hogy a véletlenszerű generálás és a kártyakezelés összhangban van, biztosítva az elvárt stratégiai mélységet és újrajátszhatóságot.

5 ÉRTEKEZÉS

A projekt alapvető célkitűzése a tower defense és a kártyajáték mechanikák szimbiózisa volt. A fejlesztés során beigazolódott az az elméleti felvetés, miszerint a két műfaj ötvözése jelentősen növeli a játékos döntési szabadságát. Míg a hagyományos toronyvédelmi játékok gyakran válnak statikussá, a kártyarendszer bevezetése olyan dinamizmust és rövid távú döntéshozatali kényszert eredményezett, amely a tesztelés során folyamatosan fenntartotta a motivációt.

A számítások alapján a kártyák kombinációs lehetőségeinek száma (meghaladva a 198 milliárdot) elméletileg is igazolja a stratégiai mélységet, amely messze túlmutat a klasszikus, kötött pályás stratégiai játékok megoldásain.

Az egyik legfontosabb megválaszolandó kérdés az volt, hogyan kerülhető el a játék unalmassá válása. A 2D grid rendszerre épülő procedurális pályagenerálással sikeresen orvosoltuk ezt a problémát. Összehasonlítva a statikus pályatervezéssel, az általunk kidolgozott algoritmus (MapHandler.cs) minden indításkor új taktikai kihívás elé állítja a felhasználót. A waypoint-rendszer és a negyedekre osztott generálás biztosítja a játékmenet egyensúlyát. Elkerülhetők a túl rövid útvonalak, így a játékosnak mindig marad ideje reagálni a támadásokra.

A fejlesztés során dedukció útján juttottunk arra a következtetésre, hogy a kód tisztasága és a memória hatékonysága érdekében az adatokat külön kell választani a logikától. A JSON alapú tárolás és a Scriptable Objectek együttes alkalmazása (SaveLoadSystem.cs) technikai szempontból is kiemelkedő eredményt hozott. Más fejlesztői megközelítésekkel szemben, ahol az adatokat gyakran keménykódolják (hard-coding), a mi rendszerünk lehetővé teszi a játékegyensúly azonnali módosítását a programkód újrafuttatása nélkül. Ez a modularitás a gyakorlati alkalmazhatóság szempontjából kulcsfontosságú, mivel megkönnyíti a játék későbbi bővítését új kártyákkal vagy ellenségtípusokkal.

A projekt eredményeként létrejött szoftver nem csupán elméleti koncepció, hanem egy funkcionális videojáték, amely a modern Unity UI Toolkit segítségével professzionális felhasználói élményt nyújt. A kidolgozott mentési rendszer és a rezponzív GUI alkalmassá teszi a szoftvert a továbbfejlesztésre, akár mobil platformokra (Android, iOS) való portolásra is, kihasználva a Unity többplatformos támogatását.

Összességében a „Kártyavarak” sikeresen válaszolt a bevezetésben feltett kérdésekre: sikerült egy olyan rendszert alkotnunk, amely egyszerre gondolkodtató, technikailag stabil és az újrajátszhatóság révén hosszú távon is érdekes marad.

6 A FELADAT ÉRTÉKELÉSE

A „Kártyavárak – tower defense játék fejlesztése” című projekt befejezésével egy olyan komplex szoftver jött létre, amely maradéktalanul teljesíti a bevezetésben és a célkitűzésekben megfogalmazott elvárásokat. A fejlesztési folyamat során sikerült egyensúlyt teremteni a toronyvédelmi játékok taktikai mélysége és a kártyajátékok dinamikus döntéshozatala között, létrehozva egy egyedi és innovatív játékményt.

6.1 A célkitűzések teljesülésének vizsgálata

A projekt során meghatározott minden részfeladat sikeresen megvalósult.

Újrajátszhatóság: A 2D grid rendszer alkalmazásával sikerült megoldani az újrajátszhatóság elméleti problémáját. A procedurális pályagenerálás biztosítja, hogy minden játékmenet új kihívásokat tárjon a játékos elé.

Adatkezelés: A JSON alapú architektúra és a Scriptable Objectek használata nemcsak technikai stabilitást és memória optimalizálást biztosít, hanem lehetővé teszi a játékegyensúly gyors és hatékony beállítását is.

Felhasználói élmény: A Unity UI Toolkit segítségével kialakított kezelőfelület átlátható, logikusan tagolt, és minden szükséges információt közvetít a játékos felé, legyen szó a kártyák kezeléséről vagy a játék állapotának visszajelzéséről.

A projekt legnagyobb eredménye a két eltérő műfaj olyan szintű szimbiózisa, amely nemcsak technikai, hanem logikai szempontból is egy egésznek alkot. A több mint 198 milliárd lehetséges paklikombináció elméletileg is igazolja a stratégiai mélységet, amit a fejlesztés kezdetén célul tűztünk ki. A fejlesztés során alkalmazott Singleton minta és az eseményvezérelt architektúra biztosítja a kód tisztaságát és fenntarthatóságát.

6.2 Gyakorlati hasznosítás és jövőbeni perspektívák

A kidolgozott rendszer a moduláris felépítésének köszönhetően a játék könnyen bővíthető újabb kártyatípusokkal, ellenségekkel vagy pályaelemekkel. A Unity motor többplatformos támogatása révén a szoftver a jövőben mobil eszközökre is portolható, ami tovább növelheti a gyakorlati felhasználhatóságát.

ÖSSZEFOGLALÁS

A „Kártyavárak – tower defense játék fejlesztése” című projekt célja egy olyan hibrid stratégiai játék megalkotása volt, amely a toronyvédelmi és a kártyajátékok mechanikáit ötvözi az újrajátszhatóság és a stratégiai mélység növelése érdekében,. A fejlesztés folyamata a Unity játékmotorra és a C# programozási nyelvre épült, alkalmazva az objektumorientált programozás (OOP) elveit és a komponens alapú architektúrát. A munka során megvalósításra került egy 2D grid rendszeren alapuló procedurális algoritmus, amely minden alkalommal egyedi pályát és útvonalat generál, kiküszöbölve a statikus játéktérből adódó egyhangúságot. Az adatkezelést JSON alapú tartós tárolással és Scriptable Objectek alkalmazásával van optimalizálva, biztosítva a hatékony memóriakezelést és a rugalmas mentési rendszert. A fejlesztés eredményeként egy funkcionális szoftver jött létre, amely átlátható grafikus felülettel (GUI), komplex kártyarendszerrel és stabil játékmenettel rendelkezik,. A projekt sikeresen igazolta a választott műfaji szimbiózis életképességét és a technológiai megoldások hatékonyságát a stratégiai játékfejlesztésben.

RESUME

The project titled "Card Castles – Tower Defense Game Development" focuses on the creation of a hybrid strategy game that integrates the tactical elements of tower defense with the dynamic decision-making of deck-builder mechanics. The primary objective was to design a system that ensures high replayability and strategic depth through procedural content generation. Developed using the Unity Game Engine and C# programming, the software implements a 2D grid-based system to automatically generate unique maps and enemy paths for every session. For efficient data management, the architecture utilizes JSON for persistent storage and Scriptable Objects for memory optimization, allowing for flexible card statistics and a robust save-load system. The final result is a functional application featuring a responsive user interface built with the Unity UI Toolkit, offering a seamless experience across menus and gameplay phases. The project successfully demonstrates the synergy between genres, providing over 198 billion possible deck combinations and ensuring a long-lasting, challenging player experience.

Projekt s názvom „Kartové hrady – Vývoj hry Tower Defense“ sa zameriava na vytvorenie hybridnej strategickej hry, ktorá integruje taktické prvky Tower Defense s dynamickým mechanizmom ‘budovania balíka kariet’. Primárnym cieľom bolo navrhnuť systém, ktorý zabezpečí vysokú znovuhrateľnosť a strategickú hĺbku prostredníctvom generovania procedurálneho obsahu. Softvér, vyvinutý pomocou herného enginu Unity a programovania v jazyku C#, implementuje 2D mriežkový systém na automatické generovanie jedinečných máp a nepriateľských trás pre každú reláciu. Pre efektívnu správu údajov architektúra využíva JSON na trvalé ukladanie a skriptovateľné objekty na optimalizáciu pamäte, čo umožňuje flexibilné štatistiky kariet a robustný systém ukladania a načítavania. Konečným výsledkom je funkčná aplikácia s responzívnym používateľským rozhraním vytvoreným pomocou sady nástrojov Unity UI Toolkit, ktorá ponúka plynulý zážitok počas fáz hry. Projekt úspešne demonštruje súčinnosť dvoch žánrov, poskytuje viac ako 198 miliárd možných kombinácií balíčkov a zabezpečuje dlhotrvajúci a náročný hráčsky zážitok.

FELHASZNÁLT IRODALOM

- [1] J. Gregory, Game Engine Architecture, CRC Press, 2018
- [2] J. Hocking, Unity in Action, Third Edition, Manning Publications, 2022
- [3] „Unity Documentation,” Unity Technologies, 20.2.2026. [Online]. Elérhető:
<https://docs.unity3d.com/Manual/index.html>
- [4] OPENAI, *ChatGPT [online]*, San Francisco: OpenAI, 2026.
- [5] „Hivatalos C# dokumentáció,” Microsoft, [Online]. Elérhető:
<https://learn.microsoft.com/en-us/dotnet/csharp/>
- [6] „Hivatalos .NET dokumentáció,” Microsoft, [Online]. Elérhető:
<https://learn.microsoft.com/en-us/dotnet/>
- [7] „Json.NET dokumentáció,” [Online]. Elérhető:
<https://www.newtonsoft.com/json/help/html/Introduction.htm>